

PRODUCT & TECHNICAL SPECIFICATION



Stock Intelligence Platform

1. Executive Summary

A local-first, self-hosted equity & F&O research terminal

2. Product Vision & Design Principles

for Indian (NSE) and U.S. swing/positional traders — free

3. Users, Modes & Workflows

data sources, no broker integration, paper trading only.

4. Product Specification — Equity Mode

5. Product Specification — F&O Mode (India only)

Version 2.0 (post Phase 3 development, currently main branch)

6. Document date: 1 September 2026

The Shared Scanning Engine (core IP)

Stack: FastAPI (Python 3) backend + Next.js 15 / React 19 (TypeScript) frontend

7. System Architecture

Scope: India cash equities · India F&O (index & stock derivatives) · U.S. cash equities

8. Backend Specification

9. Frontend Specification

No paid APIs

No broker / order routing

Local JSON storage

Paper trading only

Not financial advice

10. Data Sources, Resilience & Storage

11. Testing & Quality Assurance

12. API Reference

13. Known Limitations & Roadmap

14. Appendix — Tech Stack & Running Locally

1. Executive Summary

The Stock Intelligence Platform is a self-hosted trading *research* terminal, not a broker. It aggregates free public market data (Yahoo Finance, NSE India, Screener.in, financial news RSS), computes a unified technical scoring model, detects chart/candlestick patterns, and turns qualifying setups into concrete, risk-defined trade plans — for both cash equities (India + U.S.) and Indian index/stock derivatives (F&O). It never places real orders; the only "trading" is a JSON-persisted paper-trading ledger.

The product runs as two cooperating services: a **FastAPI** backend that owns all data fetching, scoring, and persistence, and a **Next.js/React** frontend that renders 23 pages across two switchable modes — **Equity** (India or U.S.) and **F&O** (India only). Both scanners — the Equity Scanner and the F&O Live Scanner — are driven by one shared confluence-scoring engine and one 69-pattern detector registry, so a score means the same thing everywhere in the product.

The codebase carries its own living documentation (`SCANNERS.md` , `PRODUCT_ANALYSIS_FNO.md` , `REMAINING_WORK.md`) and a regression suite of 145+ backend (pytest) and 32+ frontend (Vitest) tests plus a Playwright mobile e2e pass. This document consolidates and reconciles those sources against the code actually mounted in `backend/main.py` and linked from the live sidebar navigation, so that superseded designs (an earlier Streamlit UI, a Claude-AI F&O advisor, a Greeks portfolio tracker, a standalone options-buying scanner) are called out explicitly rather than presented as current.

2. Product Vision & Design Principles

2.1 Vision

Give a self-directed swing/positional trader the same kind of systematic, multi-factor screening a professional desk would build in-house — entirely on free data, running on their own machine, with every score explainable down to the indicator votes and pattern rule that produced it.

2.2 Design principles

Principle	How it shows up in the product
No paid data vendors	Yahoo Finance REST chart API, NSE India public endpoints, Screener.in (HTML scrape), free news RSS. No Bloomberg/Refinitiv/broker feed, no API keys required to run the app.
Local-first	All state — watchlists, portfolio, paper trades, saved rules/strategies, universe caches — is plain JSON under <code>storage/</code> . No database server, no cloud account.
Research, not execution	No broker integration, no order routing anywhere in the codebase. The only position tracking is the paper-trading ledger.
One scoring brain, many surfaces	The Equity Scanner (India + U.S., cash) and the F&O Live Scanner (India, options) call the identical <code>confluence_scorer.py</code> + <code>pattern_registry.py</code> pipeline — see Section 6.
Fail loud, degrade gracefully	Screener.in has a circuit breaker; NSE option-chain calls fall back to a synthetic Black-Scholes chain; scans report <i>why</i> a symbol produced nothing (no data vs. insufficient bars vs. error) instead of a silent empty result.
Every number is explainable	Setup cards carry a <code>score_breakdown</code> , a <code>reasons[]</code> list, and a stated <code>rr_basis</code> for trade-plan ratios that are fixed by construction rather than measured.

2.3 Explicit non-goals

- Real-time tick streaming (all data is polled / pulled per request; SSE is used for scan *progress*, not live quotes).
- Broker connectivity or order execution of any kind.
- Multi-user accounts or authentication — this is a single-operator local tool.
- Tax reporting (STCG/LTCG/F&O P&L statements).
- Push notifications / alerting — designed (`ALERTS_DESIGN.md`) but not implemented; see Section 13.

3. Users, Modes & Workflows

3.1 Primary user

A self-directed retail swing/positional trader who already understands basic technical analysis and wants a systematic, repeatable screening workflow instead of manually charting dozens of names a day. Comfortable running a local dev server (`uvicorn` + `next dev`); not a professional software engineer.

3.2 Two modes, one region toggle

The sidebar exposes a **Market Region** switch (`IN` / `US`) and, for India, a **Market Mode** switch (`Equity` / `F&O`) that swaps the entire navigation rail between the 14-item Equity nav and the 9-item F&O nav. U.S. region only ever shows the Equity nav — there is no U.S. derivatives support.

3.3 Representative workflows

Daily equity scan (5–10 minutes)

1. **Dashboard** — read market breadth, sector heatmap, FII/DII flow for the day's bias.
2. **Equity Scanner** — run India (or U.S.) universe with default threshold 65, watch the SSE progress stream.
3. Filter the results by pattern/direction/score band; expand a card's mini-chart and backtest hit-rate.
4. **Stock Analyzer** — deep-dive the top 1–2 candidates (AI verdict, fundamentals, chart analysis).
5. **Position Sizing** — compute exact quantity at a fixed risk %.

F&O intraday/swing workflow

1. Switch to **F&O** mode → **F&O Overview** for VIX/PCR/index-futures context.
2. **Live Scanner** — pick `stocks` or `indices` universe and a timeframe set (5m...1mo); read the option plan card (strike, entry/SL/T1/T2 premium, R:R, liquidity flag).
3. **Option Chain / Expiry Heatmap** — confirm OI walls and max-pain-style context around the strike.
4. **Paper Trade** — log the position from the setup card; track P&L and win rate over time.

Strategy validation

1. **Backtest** — pick a pattern + universe + hold period; require expectancy > 0 and profit factor ≥ 1.5 before trusting a screen built on it.
2. **Rule Engine** — encode the validated filter as a reusable, saved rule set.

4. Product Specification — Equity Mode

Available in both India (**IN**) and United States (**US**) regions unless noted. Sidebar groups: Market · Analysis · Tools · Help.

Dashboard /dashboard

Market-wide situational awareness for the active region: market open/closed status, index performance cards (NIFTY 50 / BANK / IT / PHARMA / MIDCAP150 / SMALLCAP250 for India; major U.S. indices for U.S.), advance/decline breadth, sector heatmap, and FII/DII daily & 30-day net flow (India only, via **GET** `/api/market/*`).

Universe Explorer /universe

Browse any of the platform's 34 index/sector universes (19 India, 15 U.S. — see Section 6.5) with live quotes: price, change %, OHLC, 52-week high/low, 30-day and 1-year return. Backed by **GET** `/api/universe/*` and **GET** `/api/market/universe/{name}/quotes` .

News /news

Aggregated headlines from free RSS sources (Moneycontrol, Economic Times, Business Standard, LiveMint for India; an equivalent U.S. financial-news set) plus NSE corporate announcements, each run through a keyword-based sentiment classifier (POSITIVE → NEGATIVE, 5 bands). Filterable market-wide or by symbol. **GET** `/api/news/market` , `/stock/{symbol}` , `/announcements` .

Earnings /earnings

Upcoming results pulled from NSE board-meeting disclosures, plus a per-stock quarterly history (sales, net profit, OPM%, YoY growth) with an earnings-sentiment classifier derived from YoY sales+profit growth. **GET** `/api/earnings/upcoming` , `/recent/{symbol}` .

Stock Analyzer /analyzer

Single-symbol deep dive, the richest page in the product:

- **Chart** — TradingView-style candlesticks (lightweight-charts) with EMA overlays and pattern trendlines.
- **AI Analysis** — a rule-based (no LLM) composite verdict: STRONG_BUY...STRONG_SELL, a 0–100 score weighted Technical 35% + Fundamental 35% + Pattern 15% + Momentum 15%, a price target, an ATR-based stop, bull/bear narratives and risk flags. **GET** `/api/stocks/{symbol}/verdict` .
- **AI Chart Analysis** — a rule-based 14-section narrative report (trend, structure, patterns, moving averages, S/R, volume, momentum, risk management with exact entry/SL/targets, probability scores out of 10, red flags, a "smart money" read, final verdict + confidence). Fully local — no API key. **GET** `/api/stocks/{symbol}/chart-analysis` .
- **Fundamentals / Quarterly / Balance Sheet / Ratio Trends / Shareholding / Peers** — sourced from Screener.in via **GET** `/api/stocks/{symbol}/fundamentals|quarterly|balance-sheet|cash-flow|r ratios-history|shareholding|peers` .
- **Multi-timeframe breakout** — 1h/1d/1w/1mo breakout-state classification with an overall verdict, via **GET** `/api/patterns/multi-tf-breakout/{symbol}` .

Equity Scanner [/equity-scanner](#)

The primary screening surface for cash equities, India and U.S. alike. Full specification in Section 6 — this page is one of the two consumers of the shared confluence engine. Streams results over SSE from `GET /api/equity-scanner/scan/stream`; supports universe, threshold, timeframe (Daily/Weekly/Monthly), and pattern filters, with a scan-quality banner surfacing per-symbol failures.

Compare [/compare](#)

Side-by-side fundamentals and price comparison across multiple symbols (PE, PB, ROE, ROCE, OPM, D/E, growth rates) plus an overlaid price chart.

Backtest [/backtest](#)

Strategy-first historical pattern testing: pick a chart pattern (from the pattern registry / legacy 39-pattern library exposed via `GET /api/patterns/library`) and a universe; the engine walks forward through history, simulates target/stop/time-stop exits, and reports win rate, expectancy, profit factor, max drawdown, an equity curve, and a full trade log. `POST /api/backtests/run` and the streaming variant `/run/stream`.

Position Sizing [/position-sizing](#)

Three sizing modes, all served by `backend/routers/positions.py`:

- **Fixed-fraction** (`POST /api/positions/calc`) — capital, risk % per trade, entry/stop/target → exact share quantity, capital deployed, capital at risk, R:R, with warnings for poor R:R or over-risking.
- **Kelly Criterion** (`POST /api/positions/kelly`) — win rate + avg win/loss % → full/half/quarter Kelly allocation; flags a negative edge outright.
- **AI Capital Allocator** (`POST /api/positions/ai-allocate`) — scores a universe, picks the top N by AI score, and allocates capital weighted by score (linear / score-squared / equal), capped per position.

Rule Engine [/rules](#)

Visual, no-code screener builder: up to 10 conditions per rule (field × operator × value, operators `> ≥ < ≤ == between`), AND/OR logic within and across rules. Rules persist to `storage/config/rules.json`. `GET/POST/DELETE /api/rules`, apply via `POST /api/rules/apply`.

Portfolio [/portfolio](#)

Manually-entered equity holdings tracker (symbol, qty, buy price/date) with live CMP, today's %, invested vs. current value, and P&L, backed by `storage/config/portfolio.json`. `GET/POST/DELETE /api/portfolio`.

Watchlist [/watchlist](#)

A lighter list than Portfolio (no position/qty semantics) with live price, RSI, MACD, moving averages, volume, and signal per symbol. `GET/POST/DELETE /api/watchlist`.

Reports `/reports`

Generates a full institutional-style PDF per stock — verdict box, AI score breakdown chart, a matplotlib-rendered daily candlestick chart with EMA/pattern/entry-target-stop overlays, a multi-timeframe breakout table, indicators, detected patterns, the full 14-section AI narrative, fundamentals, and recent quarters — via ReportLab (`services/pdf_report_service.py`). `POST /api/reports/generate` , `GET /api/reports/list` , `GET /api/reports/file/{filename}` .

Settings `/settings`

General preferences (refresh interval, default universe), data-source options (Screener.in session cookies, cache TTLs), and app metadata. `GET/PATCH /api/settings` .

Learn `/learn`

Self-study reference: a chart-pattern library with procedurally generated synthetic candlestick examples, technical-indicator explainers, fundamentals glossary, and risk-management guidance — `services/pattern_examples.py` served through `GET /api/patterns/library` and `/library/{name}/example` .

Orphaned legacy page. `frontend/app/scanner/` and its backend counterpart `backend/routers/scans.py` (`/api/scans/*`) implement the platform's original 12 preset fundamental/technical scans (Breakout Ready, Quality Growth, Undervalued, ...). The code still runs but the page is not linked from either sidebar — it has been superseded by the confluence-based Equity Scanner and is left in the tree unreferenced.

5. Product Specification — F&O Mode (India only)

Sidebar groups: Market · Scanners · Options · Risk · Learn, plus back-links to Dashboard/Settings.

F&O Overview /fo-dashboard

India VIX, live NIFTY/BANKNIFTY/FINNIFTY/MIDCPNIFTY index-futures cards (spot, futures LTP, basis, basis %, OI, volume, expiry), and a top-OI-spurt underlyings strip. `GET /api/fno/index-futures` , `/index-prices` , `/vix` , `/oi-spurts` .

PCR & Sentiment /pcr-sentiment

Put-Call Ratio (OI-based and volume-based) for NIFTY/BANKNIFTY with a bullish↔bearish interpretation scale and a 60-day rolling history chart (PCR + VIX), persisted daily to `storage/data/fno_history.json` . `GET /api/fno/pcr` , `/history` .

Live Scanner /live-scanner

The platform's primary active F&O tool — confluence-scored setups across 8 timeframes (5m...1mo) on the `indices` or `stocks` (live NSE F&O universe) preset, each producing an option trade plan (ATM strike, entry/SL/T1/T2 premium, lot size, liquidity flag, IV-rank warning). Full detail in Section 6. `GET /api/live-scanner/scan` , `/scan/stream` , `/chart-data` .

Option Chain /option-chain

Full CE/PE chain for any symbol and expiry (OI, OI change, volume, IV, LTP, bid/ask), with Greeks (Delta/Gamma/Theta/Vega) computed client-side per row (`frontend/lib/greeks.ts` , closed-form Black-Scholes). Falls back to a synthetic chain (see Section 10.3) when NSE is unreachable. `GET /api/fno/option-chain` .

Expiry Heatmap /expiry-heatmap

Strike × expiry OI matrix rendered as a colour-coded grid (CE and PE separately), highlighting the ATM row/column and OI concentration walls across all available expiries simultaneously — including the synthetic chain's 4–5 simulated expiries when NSE data is unavailable.

OI Analytics /oi-analytics

NSE's own OI-spurt and long/short buildup classification (Long Buildup, Short Buildup, Long Unwinding, Short Covering) as sortable, colour-coded tables. `GET /api/fno/oi-spurts` , `/oi-variations` .

Strategy Builder /strategy-builder

Multi-leg option strategy construction (spreads, straddles/strangles, iron condor, calendar) with a client-side payoff engine — `frontend/lib/strategy.ts` computes payoff diagrams, breakevens, and max profit/loss for any leg combination. Strategies can be named and saved. `GET/POST/DELETE /api/fno/strategies` . The symbol picker covers every F&O-eligible equity via `GET /api/fno/symbols` , not just indices.

Paper Trade `/paper-trade`

Virtual F&O trading ledger seeded with a fixed initial capital. Trades can be opened directly from a Live Scanner setup card (pre-filled instrument, action, strike, expiry, entry/SL/target) via the shared `PaperTradeModal` component. Tracks open/closed positions, realised P&L, a portfolio summary, and an equity curve; supports refreshing open-position LTPs from NSE. All persisted to `storage/data/paper_trades.json`. `GET /api/fno/paper-trades/{open|closed|portfolio|equity-curve}`, `POST /api/fno/paper-trades, /{id}/close`, `/refresh-prices`, `/reset`, `PATCH /{id}/price`.

Learn F&O `/fo-learn`

Static + example-driven education: options basics (CE/PE, moneyness, expiry), Greeks, common strategies, PCR/VIX interpretation, OI-buildup reading, and F&O-specific position sizing/risk rules.

Removed since earlier design docs. Three previously-built F&O features are **no longer part of the shipped product** — the frontend pages and, for two of them, the backend service modules have been deleted in a later cleanup pass, even though older internal documentation still describes them:

- **F&O AI Advisor** (`/fo-ai-advisor`, Claude-narrated trade suggestions via `fno_ai_service.py`) — page and service removed; no `/api/fno/ai-suggest` route remains.
- **Greeks Portfolio Tracker** (`/greeks`, net $\Delta/\Gamma/\Theta$ /Vega across positions via `greeks_calculator.py`) — page and service removed; no `/api/fno/greek-summary` or `/api/fno/max-pain` route remains. Per-row option Greeks now live only client-side in the Option Chain page.
- **Options Buying Scanner** (`/options-scanner`, the "Madras Trader" MTF composite scorer via `options_buying_scanner.py`) — page removed and its router is not mounted in `backend/main.py`. Its MTF alignment concept was effectively absorbed into the Equity/Live Scanner's shared confluence model (Section 6).

6. The Shared Scanning Engine (core IP)

Both scanning surfaces — the Equity Scanner and the F&O Live Scanner — are thin, universe-specific wrappers around one pipeline. This is the most heavily specified subsystem in the codebase (`SCANNERS.md`) and the part most worth getting exactly right.

6.1 Per-symbol pipeline

```
symbol
└─ get_ohlcv(symbol, interval)    one HTTP call per timeframe (Yahoo Finance REST)
    └─ add_indicators(df)         EMA20/50, VWAP, RSI14, MACD-hist, ATR14, RelVol
        └─ run_detectors(df, tf)  69 pattern detectors, timeframe-gated
            └─ score(...)          0-100 confluence score, 5 categories
                └─ threshold? keep only if total >= threshold
                    └─ build_equity_plan() | build_plan() (options)
```

6.2 Data source & resilience

All OHLCV comes from the Yahoo Finance chart REST API (`query1` / `query2` fallback), called with a fresh `requests.Session()` per call to avoid shared rate limiting. A process-level bar cache keys on (`ticker`, `interval`, `range`) with per-timeframe TTLs (60s for 5m/15m up to 6h for weekly/monthly). The final bar is dropped when it is not a fully-closed candle, so pattern detectors never fire on a still-forming bar. A scan reports what it could *not* do — `scanned` / `skipped_no_data` / `skipped_insufficient_bars` / `errors[]` / `cache_hit_rate` — so "no setups found" is distinguishable from "half the universe failed to load", surfaced in the UI as the Scan Quality banner.

6.3 Confluence scoring model (0–100)

#	Category	Max pts	What it measures
1	Trend / Structure	30	A normalised 5-way vote: price vs EMA20, price vs EMA50, EMA20/50 cross, price vs VWAP, 10-bar swing high/low.
2	Momentum	25	RSI(14) scored relative to the trend direction with a linear taper through overbought/oversold, plus MACD-histogram slope.
3	Volume confirmation	15	Relative volume vs. a 20-bar rolling average. Instruments with no volume (indices) are excluded from this category and the remaining 85 points rescaled to 100, rather than given a free neutral score.
4	Candle trigger	25	Points from the single highest-priority pattern aligned with the trend (see 6.4); a strength- ≥ 2 pattern that <i>contradicts</i> the trend instead applies a -15 penalty to the total.
5	Structural bonus	5	Flat 5 pts if any confirmed Tier-2 chart/price-action pattern is present, independent of which pattern won category 4.

Default qualifying threshold is **65** (UI slider 40–90, step 5); ≥ 80 is labelled a strong/high-confidence setup. Every card carries its own `score_breakdown` and a human-readable `reasons[]` list — there is no separate "Strong Buy/Buy/Hold" verdict string; the score plus direction (bullish/bearish/range) *is* the verdict.

6.4 Pattern registry — 69 detectors

Family	Count	Tier	Character
Candlestick	23	1	1–3 bar Japanese candle shapes (Hammer, Engulfing, Morning/Evening Star, ...)
Price action	12	1 (11) / 2 (1)	Bar structure & location context (Inside Bar, NR7, Breakout Retest, 52W proximity, ...)
Volume	7	1	Volume-relative signals (Volume Surge, Accumulation/Distribution, Selling Climax, ...)
Chart	25	2	Classic multi-bar formations (Head & Shoulders, Double/Triple Top-Bottom, Flags, Wedges, Triangles, Cup & Handle, Channels, Trendlines, ...)
Harmonic/Wyckoff	2	2	Accumulation/Distribution range context — low confidence, treated as flags not triggers.

Every pattern is timeframe-gated (chart patterns never fire on `5m`; the four 52-week/DMA pullback patterns only fire on Daily/Weekly/Monthly) and carries a stable `pattern_id` so directional variants (Breakout/Breakdown) nest under one parent for filtering. When the registry finds nothing, a legacy 10-pattern single-bar fallback runs.

`best_for_confluence` selects exactly one pattern per card to feed category 4, preferring a confirmed structural pattern over a candlestick over a volume-only signal.

6.5 Universes

Scanner	Regions	Universe count	Timeframes	Instrument
Equity Scanner	India + U.S.	34 (19 India, 15 U.S.) — indices, broad market, sector baskets, and full-exchange <code>all_nse</code> / <code>all_us</code>	Daily · Weekly · Monthly	Cash stock
F&O Live Scanner	India only	2 (<code>indices</code> : 6 index derivatives; <code>stocks</code> : live NSE F&O equities, default)	5m · 15m · 30m · 1h · 4h · 1d · 1wk · 1mo	Option (CE/PE)

India ↔ U.S. is purely a ticker-resolution and currency/locale switch (`.NS` suffix vs. bare ticker, ₹ vs \$); scoring, patterns, and thresholds are identical on both sides.

6.6 Trade-plan builders

Equity plan — `equity_advisor.py`

```
risk = max(ATR × 1.5, spot × 0.003)
BUY:  entry=spot, SL=spot-risk
      T1=spot+1.5*risk, T2=spot+2.5*risk
SELL: mirrored
```

R:R is fixed at 1.5 by construction, not measured — the plan states this via `rr_basis`. Range-direction setups get no plan.

Option plan — `option_advisor.py`

```
strike = ATM (spot rounded to interval)
Directional: BUY CE (bullish) / BUY PE (bearish)
Range + sufficient IV history: OTM premium-sell
Premium legs ~ 0.5 (ATM delta) × spot move
```

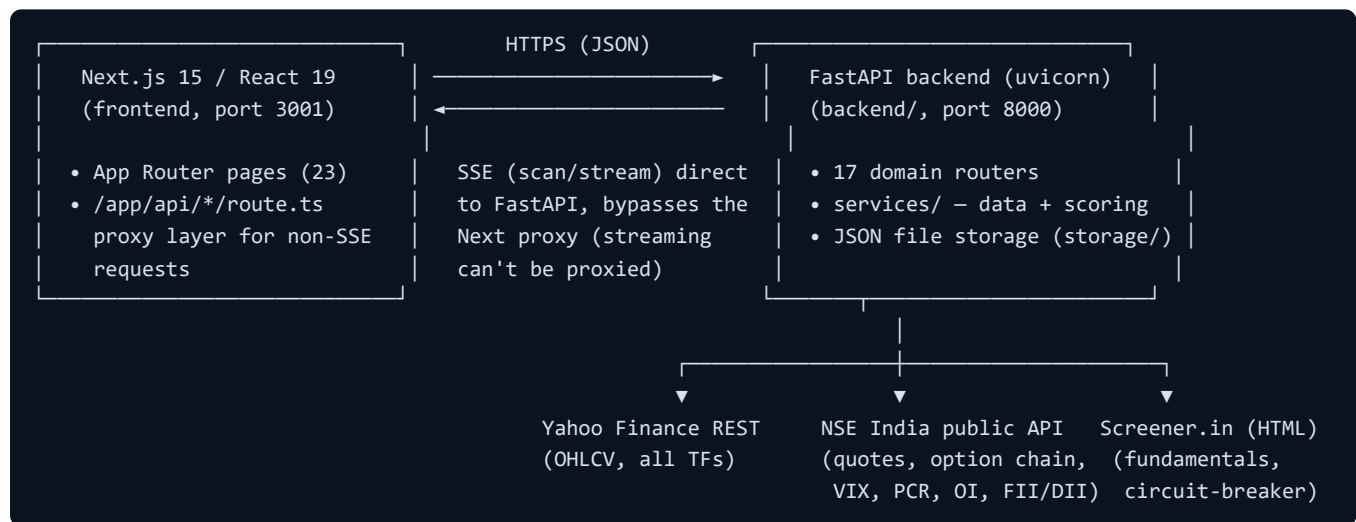
Strike interval, lot size (`services/lot_sizes.py`, single source of truth, generated into `frontend/lib/lot-sizes.generated.ts`), and a liquidity check (OI ≥ 500, volume ≥ 100) are all applied; liquidity failure is reported, not enforced.

6.7 Per-card backtest

Every card with a named pattern is backtested on its own timeframe: the registry detector walks expanding windows over the trailing 260 bars, entering at close with the same ATR risk model, and calling a "win" when T1 (1.5×risk) is

touched within 5 bars before the stop. A hit rate is only reported when the sample is ≥ 5 occurrences; otherwise the card shows the reason (in-sample, unweighted — a research aid, not a guarantee).

7. System Architecture



There is no message queue, no cache server, and no database — the backend is the single source of truth and every piece of state is either fetched fresh (with the process-level bar cache), computed on demand, or read/written as JSON under `storage/`.

8. Backend Specification

8.1 Application shell (`backend/main.py`)

- **CORS** — wide-open (`allow_origin_regex` , credentials on) since this is a local single-user tool.
- **Conditional GZip** — compresses all responses except paths ending in `/stream` , since gzip buffering would break SSE's real-time flush.
- **Global exception handler** — every uncaught exception returns a structured 500 (`type` , `message` , `path` , last 15 lines of traceback) instead of an opaque 500.
- **Startup lifespan** — enlarges FastAPI's default thread pool (8 workers) so paper-trade/health endpoints are never starved by scanner workloads; asserts no U.S.-universe symbol resolves to a `.NS` ticker (would be a silent per-symbol outage); kicks off a background universe sync.
- **Structured logging** — stdout + rotating file at `storage/logs/backend.log` (5 MB × 3 backups); slow (>1s) or error requests are logged with timing.
- **Ops endpoints** — `GET /api/health` , `GET /api/_logs/tail` (tail the log file when terminal output isn't visible).

8.2 Routers (17, all under `backend/routers/`)

Router	Prefix	Responsibility
market.py	/api/market	Market status, index/sector performance, FII/DII, universe quotes
stocks.py	/api/stocks	Per-symbol quote, OHLCV, indicators, fundamentals, quarterly/BS/CF, ratios history, shareholding, peers, AI verdict, AI chart analysis, snapshot
patterns.py	/api/patterns	Pattern library (Learn), single-symbol detection, multi-TF breakout, pattern scan (+stream)
scans.py	/api/scans	Legacy 12-preset fundamental/technical scanner (orphaned from nav, still functional)
news.py	/api/news	Market/stock news, NSE announcements
earnings.py	/api/earnings	Upcoming results, recent quarterly earnings + sentiment
positions.py	/api/positions	Fixed-fraction sizing, Kelly, AI capital allocation
backtests.py	/api/backtests	Pattern backtest (+stream)
portfolio.py	/api/portfolio	Equity holdings CRUD + live P&L
watchlist.py	/api/watchlist	Equity watchlist CRUD
rules.py	/api/rules	Rule CRUD + apply-to-universe
universe.py	/api/universe	Universe listing/search/symbols, sync trigger, sync status
settings.py	/api/settings	App settings get/patch
reports.py	/api/reports	PDF report generate/list/download
fno.py	/api/fno	Option chain, futures, VIX, PCR, OI spurts/variations, F&O symbols, watchlist, underlying quotes, IV rank/snapshot, PCR/VIX history, saved strategies, paper trades
live_scanner.py	/api/live-scanner	F&O confluence scan (+stream), universe, chart-data, patterns, health
equity_scanner.py	/api/equity-scanner	Equity confluence scan/stream, patterns, universe, universe refresh, chart-data, health

8.3 Services layer (services/ , 27 modules)

Module	Role
intraday_data.py	Yahoo Finance fetch, ticker resolution (index map / equity overrides / U.S. as-is / .NS default), universe presets, timeframe→range map, bar cache, incomplete-bar guard
confluence_scorer.py	Indicator computation + the 5-category 0–100 score (Section 6.3)
pattern_registry.py	The 69-detector modern pattern engine
pattern_detector.py	10-pattern legacy single-bar fallback detector
scan_support.py	Input validation, pattern post-filtering, per-card backtest, scan diagnostics
lot_sizes.py	Single source of truth for F&O lot sizes (generated into the frontend)
equity_advisor.py / option_advisor.py	ATR-based cash and options trade-plan builders
breakout_service.py	Per-candle breakout-state classifier (verge/fresh/confirmed/extended)
ai_service.py	Rule-based composite AI verdict (Technical/Fundamental/Pattern/Momentum weighted score)
chart_analysis_service.py	Rule-based 14-section institutional chart narrative
position_sizing_service.py	Fixed-fraction, Kelly, and AI-weighted allocation math
backtest_service.py	Walk-forward pattern backtest simulator
earnings_service.py / news_service.py	NSE board-meeting + Screener earnings; RSS news + NSE announcements; sentiment classifiers
screener_service.py	Screener.in scraper with a 3-failure/5-minute circuit breaker
nse_service.py	Thread-safe NSE session (cookie refresh every 5 min), market status/indices/sectors/FII-DII/quotes
fno_data_service.py	NSE option chain, futures, PCR, OI endpoints, IV history/rank, synthetic Black-Scholes chain fallback
mtf_analyzer.py	Multi-timeframe (monthly/weekly/daily/hourly) trend & structure analysis, used by pattern/backtest tooling
paper_trade_service.py	Thread-safe JSON-persisted paper-trading ledger
pattern_examples.py	Procedurally generated synthetic OHLCV for the Learn pattern gallery
pdf_report_service.py	ReportLab + Matplotlib PDF report assembly
universe_sync.py / us_universe_sync.py	Sync NSE / U.S. exchange index constituents to local JSON
us_market_service.py / us_news_service.py	U.S.-region equivalents of the NSE market and news services
market_router.py	Shared IN/US routing helpers used across market-data services
tradingview_service.py	Backward-compatibility shim re-exporting market-data functions under legacy names

8.4 Concurrency model

Each scanner owns a dedicated thread pool so neither starves the other or the default FastAPI pool used by health checks and paper trades: Equity Scanner uses `max_workers=16` with a symbol semaphore of 10; F&O Live Scanner

uses `max_workers=12` with a symbol semaphore of 8 (blocking) / 12 (SSE stream). A scan therefore issues roughly 10 symbols × up to 8 timeframes in flight at once for F&O, or 10 symbols × 3 timeframes for equity.

8.5 Streaming (SSE)

Both scanners expose a `/scan/stream` endpoint that the browser calls directly (bypassing the Next.js proxy, since streaming responses can't be proxied through a route handler cleanly). Event sequence: `start` (echoes the *resolved* universe/timeframes/threshold/pattern_mode) → repeated `progress` (per-symbol status: ok/no_data/insufficient_bars/error) → `enriching` (option chain + backtest enrichment begins) → `result` (full payload + summary) → literal `[DONE]`.

9. Frontend Specification

9.1 Stack

Library	Version	Purpose
Next.js	15.1.3	App Router, dev server on port 3001, file-based routing for the 23 pages
React	19.0.0	UI runtime
TypeScript	^5.7.2	Type safety across API clients and components
Tailwind CSS	^4.0.0	Utility-first styling (dark, Zerodha-inspired theme via CSS variables)
Radix UI	various	Accessible primitives — dialog, dropdown, popover, select, tabs, toast, tooltip
@tanstack/react-query	^5.62.7	Server-state fetching/caching for all non-streaming API calls
@tanstack/react-table	^8.20.6	Sortable/filterable data tables (scanner results, chains, watchlist)
lightweight-charts	^4.2.2	TradingView's open-source charting engine — candlesticks, EMAs, price lines, pattern overlays
recharts	^2.15.0	General analytics charts (equity curves, breadth, sentiment, Greeks)
lucide-react	^0.469.0	Icon set for the sidebar and controls
Vitest / Playwright	^2.1.8 / 1.49.1	Unit and mobile-viewport e2e testing

9.2 Structure

- `frontend/app/` — one directory per route (23 pages) plus `app/api/*/[...slug]/route.ts` catch-all proxies that forward non-streaming requests to FastAPI (a documented, load-bearing rule — the F&O proxy must forward to FastAPI, never to NSE directly, or every F&O page that expects the normalised backend response shape silently breaks).
- `frontend/components/` — `layout/` (Sidebar, region/mode switches), `scanner/` (setup cards, CandleChart, PatternPicker, MultiSelectFilter, ScanQualityBanner, PaperTradeModal), `charts/`, `common/`, `ui/` (Radix-based primitives).
- `frontend/lib/` — typed API clients (`api.ts` , `fno-api.ts`), domain types (`scanner-types.ts` , `equity-scanner-types.ts` , `fno-types.ts`), client-side calculation modules (`greeks.ts` , `strategy.ts`), `market-hours.ts` , region/nav config, generated lot sizes.

9.3 Region & mode switching

A React context (`lib/region.tsx`) drives a two-level sidebar: Market Region (IN/US) selects the data universe and currency locale; within IN, Market Mode (Equity/F&O) swaps the entire nav array (`EQUITY_NAV` vs. the config-driven `FO_NAV` built from `lib/nav-config.ts`). Navigating directly to an F&O URL auto-switches the mode via a prefix match against `FO_PREFIXES` .

10. Data Sources, Resilience & Storage

10.1 Data sources

Source	What's fetched	Resilience
Yahoo Finance REST chart API	OHLCV at every scanner timeframe, for both India and U.S. tickers	query1→query2 fallback, fresh session per call, process-level bar cache with per-TF TTL
NSE India public API	Live quotes, option chain, futures, VIX, PCR inputs, OI spurts/variations, index constituents, FII/DII, board meetings, announcements	Persistent session with cookies refreshed every 5 min; 401/403 forces immediate re-seed; synthetic Black-Scholes chain fallback when unavailable
Screener.in (HTML scrape)	Fundamentals, quarterly/annual P&L, balance sheet, cash flow, shareholding history, peers	Circuit breaker opens after 3 consecutive failures for 5 minutes; chart/technicals keep working even when this is down
Free news RSS + NSE announcements	Market and stock-specific headlines	Parallel fetch, de-duped by title

10.2 Symbol resolution

Ticker resolution order: (1) explicit index map (NIFTY→`^NSEI`, BANKNIFTY→`^NSEBANK`, SENSEX→`^BSESN`, MIDCPNIFTY→`NIFTY_MID_SELECT.NS`, BANKEX→`BSE-BANK.BO`, U.S. indices to their own tickers); (2) a small equity-override table (e.g. TATAMOTORS→`TMCV.NS`, ETERNAL→`ETERNAL.NS`, following the Zomato→Eternal rename); (3) any symbol present in a `storage/universe/US_*.json` file is treated as U.S. and used bare, no suffix; (4) default — append `.NS`. A startup assertion checks no U.S.-universe symbol still resolves to a `.NS` ticker, turning what would otherwise be a silent per-symbol outage into a logged error.

10.3 Synthetic option chain fallback

When NSE's option-chain endpoint is unreachable, the backend generates a full synthetic chain: ATM strike from the symbol's interval, ± 20 strikes, an IV skew model ($IV = ATM_IV \times \exp(-0.4 \times \text{moneyness})$), closed-form Black-Scholes premiums at a 6.5% risk-free rate, a Gaussian OI curve peaked one strike OTM with a round-strike multiplier, and realistic weekly/monthly expiry dates per instrument. The response is flagged `synthetic: true` so the UI can show a disclaimer; this keeps the option-plan and expiry-heatmap features functional even during an NSE outage.

10.4 Storage layout (all JSON, no database)

Path	Contents
<code>storage/cache/</code>	TTL-controlled OHLCV/fundamentals cache files
<code>storage/data/</code>	<code>paper_trades.json</code> , <code>fno_history.json</code> (60-day PCR+VIX), <code>iv_history.json</code> (per-symbol ATM IV, up to 260 daily snapshots), <code>saved_strategies.json</code>
<code>storage/config/</code>	<code>watchlist.json</code> , <code>fno_watchlist.json</code> , <code>portfolio.json</code> , <code>rules.json</code> , app settings
<code>storage/universe/</code>	Cached index/sector constituents for all 34 universes (India + U.S.)
<code>storage/reports/</code>	Generated PDF research reports
<code>storage/logs/</code>	Rotating backend log

11. Testing & Quality Assurance

Suite	Tooling	Coverage focus
Backend unit/integration	pytest, pytest-asyncio, respx (HTTP mocking)	145+ tests across confluence scoring, pattern registry, trade-plan math, lot-size drift, IV rank, scan diagnostics, and every Phase 3 fix (ticker resolution, max-pain math prior to its removal, scanner routing)
Frontend unit	Vitest	32+ tests — scanner routing/type guards, formatting helpers, region/nav config invariants
Mobile e2e	Playwright (<code>mobile-380</code> project, 380×812 viewport)	9 tests confirming no horizontal overflow and control visibility on Option Chain, (formerly) Greeks, and Expiry Heatmap at phone width

A dedicated regression file (`tests/test_pattern_registry.py` , plus new `test_confluence_scoring.py` and `test_scan_data_layer.py`) locks in the exact scoring and pattern-matching guarantees described in Section 6 — including the "a pattern filter never changes a setup's score" contract. Run everything deterministic with `make test` ; Playwright separately via `make test-playwright` (requires a running Next.js dev server).

12. API Reference (condensed — grouped by router, verified against mounted code)

12.1 Equity & market data

Method	Path	Purpose
GET	<code>/api/market/status</code>	Open/closed, index level, market cap
GET	<code>/api/market/indices</code>	Headline index performance cards
GET	<code>/api/market/sectors</code>	Sector index performance + breadth
GET	<code>/api/market/fii-dii</code>	FII/DII daily flows (India)
GET	<code>/api/market/universe/{name}/quotes</code>	Full quote table for a universe
GET	<code>/api/stocks/{symbol}/quote ohlcv indicators fundamentals quarterly balance-sheet cash-flow ratios-history shareholding peers snapshot</code>	Per-symbol data surfaces backing the Stock Analyzer
GET	<code>/api/stocks/{symbol}/verdict</code>	Rule-based AI composite verdict
GET	<code>/api/stocks/{symbol}/chart-analysis</code>	14-section chart narrative
GET	<code>/api/universe/list \ search \ all-symbols \ {name} \ {name}/symbols</code>	Universe metadata and membership
POST	<code>/api/universe/{name}/sync</code>	Rebuild one universe from NSE/exchange source
GET	<code>/api/universe/_meta/status</code>	Sync status for all universes

12.2 Scanning

Method	Path	Purpose
GET	<code>/api/equity-scanner/scan/stream</code>	SSE confluence scan, India + U.S. cash equities
GET	<code>/api/equity-scanner/patterns \ universe \ universe-count/{key} \ chart-data \ health</code>	Supporting metadata for the Equity Scanner UI
POST	<code>/api/equity-scanner/refresh-universe/{all_nse\ all_us}</code>	Rebuild the full-exchange universe
GET	<code>/api/live-scanner/scan</code>	Blocking F&O confluence scan
GET	<code>/api/live-scanner/scan/stream</code>	SSE F&O confluence scan
GET	<code>/api/live-scanner/universe \ chart-data \ patterns \ health</code>	Supporting metadata for the Live Scanner UI
GET	<code>/api/scans/types</code> , POST <code>/run</code> , POST <code>/run/stream</code>	Legacy 12-preset scanner (orphaned page)
GET	<code>/api/patterns/library \ library/{name}/example \ detect/{symbol} \ multi-tf-breakout/{symbol}</code> , POST <code>/scan</code> , POST <code>/scan/stream</code>	Pattern library + legacy chart-pattern scanning

12.3 F&O data & paper trading

Method	Path	Purpose
GET	<code>/api/fno/option-chain</code>	Full CE/PE chain, synthetic fallback
GET	<code>/api/fno/index-futures \ index-prices \ vix \ pcr \ history</code>	Futures, index levels, VIX, PCR, 60-day history
GET	<code>/api/fno/oi-spurts \ oi-variations</code>	NSE OI-buildup analytics
GET	<code>/api/fno/symbols \ underlying-quotes</code>	F&O-eligible equities, live spot LTPs
GET	<code>/api/fno/scan</code>	OI-based server-side scan types
GET/POST/DELETE	<code>/api/fno/watchlist[/]{symbol}]</code>	F&O watchlist CRUD
GET/POST	<code>/api/fno/iv-rank \ iv-snapshot</code>	Per-symbol 52-week IV rank + manual snapshot trigger
GET/POST/DELETE	<code>/api/fno/strategies[/]{name}]</code>	Saved multi-leg strategies
GET	<code>/api/fno/paper-trades/portfolio \ equity-curve \ open \ closed</code>	Paper-trading ledger reads
POST	<code>/api/fno/paper-trades</code> , <code>/id/close</code> , <code>/refresh-prices</code> , <code>/reset</code>	Paper-trading ledger writes
PATCH	<code>/api/fno/paper-trades/{id}/price</code>	Manual price update on an open position

12.4 Tools & persistence

Method	Path	Purpose
POST	/api/positions/calc \ kelly \ ai-allocate	Position-sizing calculators
POST	/api/backtests/run , POST /run/stream	Pattern backtest engine
GET/POST/DELETE	/api/portfolio[/add\ /{symbol}]	Equity holdings
GET/POST/DELETE	/api/watchlist[/add\ /{symbol}]	Equity watchlist
GET/POST/DELETE	/api/rules[/id] , POST /apply	Rule engine
GET/PATCH	/api/settings	App settings
POST/GET	/api/reports/generate \ list \ file/{filename}	PDF report generation/library
GET	/api/news/market \ stock/{symbol} \ announcements	News aggregation
GET	/api/earnings/upcoming \ recent/{symbol}	Earnings calendar & history

13. Known Limitations & Roadmap

13.1 Standing limitations (by design or unresolved)

- **No real-time streaming quotes.** All market data is pulled on demand; SSE is used only for scan progress, not live ticks.
- **No broker integration.** Deliberate — the product is a research/paper-trading tool.
- **Liquidity is reported, not enforced** on option plans — a plan with `liquidity_ok: false` is still returned with a warning.
- **R:R ratios on trade plans are fixed design parameters** (1.5 directional, 0.33 premium-sell), not measurements from swing structure — every plan states this via `rr_basis`.
- **Backtest hit rates are in-sample and unweighted**, over a 260-bar trailing window per symbol; small samples (<5) are suppressed rather than shown as misleadingly precise.
- **Lot sizes need periodic manual reconciliation** against NSE's live contract master — the Python↔TypeScript pair is drift-tested, but the underlying values are not auto-synced.
- **The default confluence threshold (65) has drifted loose** after recent scoring corrections (trend normalisation, VWAP fix, RSI-band fix) shifted the whole score distribution upward — a documented, open product decision to raise it (recommendation: 75) rather than a bug.
- `all_nse` / `all_us` universes require a manual **Refresh** before first use.
- **Historical F&O data** (NSE ban list, participant-wise OI, rollovers) is out of scope — would require Bhav Copy batch downloads or a paid vendor.
- **Screener.in scraping is inherently fragile** to upstream HTML changes; the circuit breaker limits blast radius but cannot prevent staleness.

13.2 Descoped since earlier design phases

See Section 5's note — the F&O AI Advisor (Claude-narrated suggestions), the Greeks Portfolio Tracker (incl. the max-pain endpoint), and the standalone Options Buying Scanner have all been removed from the shipped product since they were originally built and documented. Any future work referencing those pages should first confirm whether they are being reintroduced or definitively retired.

13.3 Designed but not built

Alerts engine — `ALERTS_DESIGN.md` specifies 5 v1 rule types (scanner score, paper-trade SL/TP, VIX threshold, PCR threshold) delivered in-app over the existing SSE infrastructure, with a ~20-hour v1 effort estimate and Telegram delivery flagged for a v2 decision. Awaiting product sign-off before implementation.

14. Appendix — Tech Stack & Running Locally

14.1 Backend dependencies (selected, from requirements.txt)

Library	Version	Role
fastapi	0.115.6	API framework
uvicorn[standard]	0.34.0	ASGI server
pandas / numpy	2.2.3 / 2.2.4	OHLCV manipulation
ta	0.11.0	Technical indicator library (RSI, MACD, ADX, BB, Stochastic, CCI, ATR)
scipy	≥1.11.0	Black-Scholes / statistical helpers
yfinance / nselib	≥0.2.40 / ≥2.5.0	Market data client libraries
beautifulsoup4 / lxml	4.13.3 / 5.2.1	Screener.in HTML scraping
reportlab / matplotlib	4.3.1 / ≥3.7	PDF report generation
feedparser	6.0.11	News RSS parsing
anthropic	≥0.40.0	Retained dependency from the (since-removed) Claude F&O advisor; not currently invoked by any mounted route
pytest / pytest-asyncio / respx	≥8.0.0 / ≥0.24.0 / ≥0.21.0	Test suite

14.2 Running the stack locally

```
# Backend
pip install -r requirements.txt
uvicorn backend.main:app --reload --port 8000

# Frontend (separate terminal)
cd frontend
pnpm install
pnpm dev          # next dev -p 3001

# Tests
make test          # backend pytest + frontend vitest
make test-playwright  # mobile e2e (needs the Next.js dev server running)
```

Frontend talks to FastAPI at NEXT_PUBLIC_API_URL (default http://127.0.0.1:8000) both through the Next.js proxy route handlers and, for F&O AI/paper-trade calls and all SSE scans, directly from the browser.

This document was compiled by reconciling the platform's internal documentation (SCANNERS.md , PRODUCT_ANALYSIS_FNO.md , CHANGELOG_PHASE2/3.md , REMAINING_WORK.md) against the code actually mounted in backend/main.py and linked from the live sidebar navigation, on 1 September 2026. The platform is a research and education tool. Nothing in this document, or in the product it describes, constitutes financial advice — all scores, patterns, and trade plans are algorithmically generated and unverified against live market outcomes.